FORGE & FADE
Interview Presentation Guide

A practical speaking script and code walkthrough for a full-stack React and Laravel booking application.

LIVE PROJECT
kamva-hanisi.github.io/Forge-And-Fade/

Prepared from the working Forge & Fade repository

This guide is designed for a practical interview presentation of about 7-10 minutes. Do not try to memorize every sentence. Learn the flow, explain it in your own words, and use the code references when the interviewer wants more detail.

1. Short opening

What to say

> This project is called Forge & Fade. It is a full-stack barbershop website that allows customers to explore services and barbers, check appointment availability, make a booking, and send an enquiry. I built the frontend with React and Vite, and I built the REST API with Laravel. The frontend is deployed on GitHub Pages, while the API is deployed separately on Render. The two applications communicate using JSON over HTTP. The main problem I wanted to solve was making the booking process simple for the customer while preventing two customers from booking the same barber at the same date and time.

Live website: <https://kamva-hanisi.github.io/Forge-And-Fade/>

Live API base: <https://forge-and-fade.onrender.com/api>

2. Project overview

What to show

Start on the live home page. Briefly show:

1. The home page and responsive navigation.
2. The services section.
3. The barber profiles.
4. The booking page.
5. The contact page.

What to say

> The public pages give the customer the information they need before booking. The services and barber data are reusable frontend data, so the same information can be displayed on multiple pages without repeating the markup. A customer can also start a booking from a specific service or barber. The link adds the selected ID to the URL, and the booking form reads that value and preselects it.

Code to show

- `frontend/src/data/services.js` - reusable service names, prices, descriptions, and durations.
- `frontend/src/data/barbers.js` - reusable barber profiles.
- `frontend/src/components/ServicesSection/ServicesSection.jsx:41` - passes a service through the booking URL.
- `frontend/src/components/BarbersSection/BarbersSection.jsx:48` - passes a barber through the booking URL.
- `frontend/src/pages/Booking/Booking.jsx:28` - reads the URL parameters.

```
const [searchParams] = useSearchParams();

const serviceFromUrl = searchParams.get("service") || "";
const barberFromUrl = searchParams.get("barber") || "";
```

3. High-level architecture

What to say

> I separated the system into a client and an API. React is responsible for the interface, page navigation, form state, and feedback to the user. Laravel is responsible for trusted validation, business rules, and database access. The browser never connects directly to the database. It sends an HTTP request to the Laravel API. Laravel validates the request, runs the booking logic, uses an Eloquent model to access the database, and then returns a JSON response to React.

```
Customer
   |
   v
React frontend on GitHub Pages
   |
   | Axios HTTP request with JSON
   v
Laravel API on Render
   |
   | Validation + business rules + Eloquent
   v
Database
   |
   | JSON response
   v
React updates the interface
```

Important clarification

Say **the frontend calls the backend**, not "the backend connects to the frontend." The backend responds to API requests from approved frontend origins. CORS controls which browser origins may make those requests.

---

4. How the frontend connects to the backend

What to say

> The frontend gets the API base URL from a Vite environment variable called `VITE_API_URL`. This allows local development and production to use different backend URLs without changing the component code. I use Axios for the requests. For example, once the customer selects a barber and date, React sends a GET request to the availability endpoint. When the customer confirms the form, React sends a POST request containing the booking data.

Code to show

Open `frontend/src/pages/Booking/Booking.jsx` around lines 52-88.

```
const apiUrl =
  import.meta.env.VITE_API_URL || "http://127.0.0.1:8000/api";

const response = await axios.get(`${apiUrl}/bookings/availability`, {
  params: {
    barber_id: formData.barber_id,
    booking_date: formData.booking_date,
  },
});

setBookedTimes(response.data.booked_times || []);
```

Then show the POST request around lines 102-149.

```
const response = await axios.post(`${apiUrl}/bookings`, {
  ...formData,
  service_id: selectedService.id,
  service_name: selectedService.name,
  barber_id: selectedBarber.id,
```

```
    barber_name: selectedBarber.name,
    duration: selectedService.duration,
  });
```

Explain the React state

> The form is controlled by React state. Each input gets its value from `formData`, and `handleChange` updates the correct property. I also keep separate state for loading, errors, success messages, booked times, and the confirmed booking. This lets the interface clearly show what is happening during an asynchronous request.

Useful lines:

- `frontend/src/pages/Booking/Booking.jsx:33` - form state.
- `frontend/src/pages/Booking/Booking.jsx:44` - request and feedback state.
- `frontend/src/pages/Booking/Booking.jsx:90` - shared input change handler.
- `frontend/src/pages/Booking/Booking.jsx:141` - API error handling.

---

5. How availability works

What to say

> Availability is checked when the selected barber or date changes. The `useEffect` hook requests the already-booked times for that exact barber and date. The response is stored in `bookedTimes`, and matching time options are disabled in the form. This improves the user experience, but I do not rely only on disabled options in the browser. A user could send a request manually, or another customer could take the slot before the first customer submits. For that reason, the backend checks the booking again.

Code to show

Frontend filtering in `frontend/src/pages/Booking/Booking.jsx:244`:

```
{timeSlots.map((time) => {
  const unavailable = bookedTimes.includes(time);

  return (
    <option value={time} key={time} disabled={unavailable}>
      {time}{unavailable ? " - Booked" : ""}
    </option>
  );
})}
```

API route in `backend/routes/api.php:8`:

```
Route::get(
    '/bookings/availability',
    [BookingController::class, 'availability']
);
```

Backend query in `backend/app/Http/Controllers/Api/BookingController.php:71`:

```
$bookedTimes = Booking::where('barber_id', $validated['barber_id'])
    ->whereDate('booking_date', $validated['booking_date'])
    ->where('status', '!=', 'cancelled')
    ->pluck('booking_time');
```

> Laravel uses the Booking model to query only active appointments for that barber and date. It returns a small JSON response containing only the booked times needed by the interface.

---

6. How a booking is created

What to say

> The POST route sends the request to the controller's `store` method. The controller first validates every field. This is important because browser validation improves usability, but server-side validation is the real security and data-integrity boundary.

Code to show

Route in `backend/routes/api.php:13`:

```
Route::post('/bookings', [BookingController::class, 'store']);
```

Validation in `backend/app/Http/Controllers/Api/BookingController.php:11`:

```
$validated = $request->validate([
    'service_id' => 'required|integer',
    'barber_id' => 'required|integer',
    'booking_date' => 'required|date|after_or_equal:today',
    'booking_time' => 'required|date_format:H:i',
    'customer_name' => 'required|string|max:100',
    'email' => 'required|email|max:150',
    'phone' => 'required|string|max:30',
    'notes' => 'nullable|string|max:1000',
]);
```

Duplicate check in the same controller around line 32:

```
$alreadyBooked = Booking::where('barber_id', $validated['barber_id'])
    ->whereDate('booking_date', $validated['booking_date'])
    ->where('booking_time', $validated['booking_time'])
    ->where('status', '!=', 'cancelled')
    ->exists();

if ($alreadyBooked) {
    return response()->json([
        'message' => 'That time is already booked. Please choose another time.',
    ], 409);
}
```

Creation and response around line 58:

```
$booking = Booking::create([
    ...$validated,
    'status' => 'confirmed',
]);

return response()->json([
    'message' => 'Your appointment has been booked successfully.',
    'booking' => $booking,
], 201);
```

HTTP statuses to mention

- `201 Created` means a booking was successfully created.
- `409 Conflict` means the requested slot is already occupied.
- `422 Unprocessable Entity` is returned automatically by Laravel when validation fails.

7. How the data reaches the database

What to say

> Laravel's Eloquent ORM connects the controller to the database. The Booking model defines which fields may be mass assigned. The migration defines the actual table structure.

Code to show

Model: `backend/app/Models/Booking.php:7`

```
class Booking extends Model
{
    protected $fillable = [
        'service_id',
        'service_name',
        'barber_id',
        'barber_name',
        'booking_date',
        'booking_time',
        'duration',
        'customer_name',
        'email',
        'phone',
        'notes',
        'status',
    ];
}
```

Database protection:
`backend/database/migrations/2026_09_25_005755_create_bookings_table.php:35`

```
$table->unique(
    ['barber_id', 'booking_date', 'booking_time'],
    'unique_barber_booking'
);
```

Strong interview point

> Double-booking is handled at three levels. The frontend disables known occupied times for usability. The backend checks for a conflict before creating the record. Finally, the database has a unique constraint on barber, date, and time as the last line of protection.

Be honest about one limitation: because the unique constraint includes cancelled rows, a cancelled slot cannot currently be reused without updating or removing that old record. This is a good future improvement to discuss.

---

8. Contact form flow

What to say

> The contact form uses the same client-server pattern. React sends the form data to `/api/contact`. Laravel validates it, stores it through the ContactMessage model, sets its initial status to `new`, and returns a `201` response.

Code to show

- `frontend/src/pages/Contact/Contact.jsx:33` - form submission.
- `backend/routes/api.php:18` - contact API route.
- `backend/app/Http/Controllers/Api/ContactController.php:11` - validation and creation.
- `backend/app/Models/ContactMessage.php:7` - database model.
- `backend/database/migrations/2026_09_25_010131_create_contact_messages_table.php:11` - table definition.

---

9. Calendar integration

What to say

> After a successful booking, the API returns the saved appointment. The frontend uses that confirmed data to offer calendar actions. Google Calendar is opened with URL parameters, while Apple Calendar receives a generated `.ics` file. The end time is calculated from the service duration.

Code to show

- `frontend/src/pages/Booking/Booking.jsx:327` - calendar buttons appear only after confirmation.
- `frontend/src/utils/calendar.js:13` - calculates start and end times.
- `frontend/src/utils/calendar.js:56` - builds the Google Calendar URL.
- `frontend/src/utils/calendar.js:83` - creates and downloads the ICS file.

---

10. Routing and GitHub Pages

What to say

> The application uses React Router. I chose `HashRouter` because GitHub Pages serves static files and does not automatically rewrite routes back to `index.html`. Hash-based routes such as `#/booking` work even when the page is refreshed directly.

Code to show

`frontend/src/main.jsx:8`:

```
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <HashRouter>
      <App />
    </HashRouter>
  </StrictMode>,
);
```

`frontend/src/App.jsx:19` contains the page routes and shared layout components.

---

11. CORS and environment configuration

What to say

> Because the frontend and backend are hosted on different origins, the backend must explicitly allow the frontend origin. Laravel's CORS configuration reads the allowed origins from an environment variable. This keeps development and production configuration separate from the source code.

Code to show

`backend/config/cors.php:3`:

```
$allowedOrigins = array_filter(array_map(
    'trim',
    explode(',', env('CORS_ALLOWED_ORIGINS', 'http://localhost:5173'))
));
```

> CORS is a browser access rule; it is not authentication. These public booking and contact endpoints do not currently require a user account.

---

12. Deployment and CI/CD

What to say

> The frontend deployment is automated with GitHub Actions. A push to `main` that changes the frontend starts the workflow. It installs exact dependencies with `npm ci`, verifies that the production API URL exists, builds the React application with the repository base path, and deploys the generated `dist` folder to GitHub Pages. The Laravel API has a production Dockerfile for Render. The image installs production PHP dependencies, configures Apache, and runs Laravel's configuration cache, route cache, and database migrations when the service starts.

Code to show

- `.github/workflows/deploy-pages.yml:3` - deployment trigger.
- `.github/workflows/deploy-pages.yml:35` - dependency installation.
- `.github/workflows/deploy-pages.yml:39` - production API URL check.
- `.github/workflows/deploy-pages.yml:48` - frontend build.
- `backend/Dockerfile:1` - multi-stage production image.
- `backend/docker/start-render.sh:9` - Laravel startup tasks.

---

13. Testing

What to say

> I added feature tests around the most important API behavior. The booking tests cover successful creation, invalid input, duplicate-slot rejection, and availability filtering. The contact tests cover successful storage and required-field validation. `RefreshDatabase` isolates each test by resetting database state.

Code to show

- `backend/tests/Feature/BookingApiTest.php:14` - successful booking.
- `backend/tests/Feature/BookingApiTest.php:38` - validation failure.
- `backend/tests/Feature/BookingApiTest.php:61` - duplicate booking conflict.
- `backend/tests/Feature/BookingApiTest.php:81` - availability result.
- `backend/tests/Feature/ContactApiTest.php:12` - contact creation.

---

14. Live demonstration script

Use this order so the demonstration tells one complete story:

1. Open the home page and say: "This is the customer-facing landing page. It introduces the services, business, and team."
2. Click a specific service's **Book** button and point out that the service is already selected on the booking page.
3. Select a barber and date. Say: "This triggers an availability request to the Laravel API."
4. Open the browser developer tools only if the interviewer asks to see the network request.
5. Explain that booked times returned by the API become disabled.
6. Do not create a fake production booking unless you intend to clean it up later. You can explain the remaining form and show the code instead.
7. Open the controller and follow `route -> validation -> conflict check -> model -> database -> JSON response`.
8. End by showing the feature test for duplicate bookings.

15. A complete 7-minute speaking script

Hi, this project is Forge & Fade, a full-stack booking website for a barbershop. A customer can browse services, choose a barber, check appointment availability, make a booking, add the confirmed appointment to a calendar, and send a contact enquiry. I used React with Vite for the frontend and Laravel for the REST API. The frontend is deployed on GitHub Pages, and the backend is deployed on Render. They are separate applications that communicate using JSON over HTTP. On the frontend I use React Router for the pages and reusable components for the shared navigation, footer, and home-page sections. The services and barber profiles are stored as reusable data. This also allows a service or barber card to link to the booking page with an ID in the URL, so the user's choice is preselected. The most important workflow is booking. The form is controlled with React state. When the customer selects both a barber and a date, a `useEffect` sends a GET request through Axios to the Laravel availability endpoint. Laravel queries bookings for that barber and date and returns the occupied times as JSON. React stores those times and disables them in the dropdown. When the customer submits the form, React sends a POST request to `/api/bookings`. The Laravel route passes that request to the BookingController. The controller validates all incoming data, including the email format, future booking date, and time format. I validate on the server because frontend validation can be bypassed. The controller then checks whether that barber, date, and time are already occupied. If they are, it returns HTTP 409 Conflict. Otherwise, Eloquent creates a confirmed booking and Laravel returns HTTP 201 Created with the saved booking as JSON. I protect against double-booking in three places. The frontend disables occupied times, the backend performs a conflict query, and the database has a unique constraint across barber, date, and time. The database constraint protects the data even if two requests arrive very close together. After the frontend receives the successful response, it displays confirmation and uses the returned booking data to create Google and Apple Calendar options. The Google action builds a calendar URL, and the Apple action generates an ICS file in the browser. The contact form follows a similar flow: React posts JSON, Laravel validates it, Eloquent stores it with a `new` status, and the API returns a success response. For deployment, GitHub Actions automatically builds and deploys the frontend when relevant code is pushed to `main`. The production API URL is injected through an environment variable. The Laravel backend uses a Docker image on Render and runs configuration caching, route caching, and migrations during startup. CORS is configured through an environment variable so only approved frontend origins can call the API from a browser. I also wrote Laravel feature tests for successful booking, validation, duplicate-booking prevention, availability filtering, and contact submissions. If I continued the project, my priorities would be an authenticated staff dashboard, appointment cancellation and rescheduling, server-side ownership of service pricing and duration, notifications, and stronger handling for simultaneous booking requests. The main lesson from this project was how to connect a separately deployed React frontend and Laravel API while keeping the important business rules and data validation on the server.

16. Likely interviewer questions

Why did you use React?

React made it straightforward to build reusable sections and a stateful booking experience. When a user changes the barber or date, the interface can fetch availability and update only the affected controls without reloading the page.

Why did you use Laravel?

Laravel provides routing, request validation, JSON responses, migrations, Eloquent, CORS support, and testing conventions. That allowed me to focus on the booking rules instead of building the API foundation from scratch.

Why is validation needed in both places?

Frontend validation gives the user quick feedback. Backend validation protects the application because requests can be created without using the frontend. The server must never trust browser input.

How do you stop double-booking?

React hides occupied options, Laravel checks the slot again when the booking is submitted, and the database has a unique constraint. The database is the final authority.

Why Axios instead of `fetch`?

Either would work. I chose Axios because it gives a concise API for query parameters, JSON requests, and error responses such as `err.response.data.message`.

Why HashRouter?

GitHub Pages does not provide custom server rewrites for this single-page application. HashRouter keeps routing on the client and prevents direct route refreshes from returning a 404.

What is CORS?

CORS is a browser security mechanism controlling whether JavaScript from one origin may call another origin. Because GitHub Pages and Render have different origins, the API must allow the frontend URL. It does not replace authentication or authorization.

Where are services and barbers stored?

They are currently static frontend data, while bookings and contact messages are stored in the database. For a larger production system, I would move services and barbers into backend tables and expose read endpoints, so staff could update them without rebuilding the frontend.

What would you improve first?

I would add a staff dashboard with authentication and authorization, then cancellation and rescheduling. I would also make the backend the source of truth for service names, prices, and durations instead of accepting those details from the client.

Is the duplicate-booking solution completely race-safe?

The pre-insert controller check gives a clear user-facing conflict message, and the unique database constraint is the final protection against two simultaneous inserts. A production improvement would be to catch the database unique-constraint exception and consistently translate it into the same 409 response. I would also revise the schema so cancelled slots can be reused safely.

How would you add authentication?

I would use Laravel Sanctum for staff authentication, protect administrative routes with authentication and authorization middleware, and keep customer booking endpoints public unless the product required customer accounts.

How did you test it?

I used Laravel feature tests because the main risk is the behavior across routing, validation, controllers, models, and the database. The tests submit real JSON requests to the application and assert both the HTTP response and stored database records.

17. Honest limitations and future work

Mentioning limitations demonstrates engineering judgment. Use only two or three unless the interviewer asks for more.

1. Services and barbers are currently frontend data. The backend should become their source of truth.
2. There is no staff dashboard for viewing, confirming, cancelling, or rescheduling appointments.
3. There are no email or SMS confirmations yet.
4. The database uniqueness design needs adjustment before cancelled time slots can be reused.
5. A database constraint exception should be converted to a friendly `409` response for simultaneous requests.
6. Frontend component and end-to-end tests could be added alongside the existing backend feature tests.
7. Production availability should account for business hours, holidays, barber schedules, service duration overlaps, and time zones.

---

18. Final closing

What to say

> Forge & Fade demonstrates the full path from a customer interaction in React to validated business logic in Laravel and persistent database storage. The part I am most proud of is that availability and double-booking are treated as backend data-integrity problems, not only as frontend form behavior. The project is deployed, tested around its core API flows, and has a clear path for production improvements.

Then stop and invite questions:

> I would be happy to go deeper into any part of the frontend, API, database design, testing, or deployment.

---

19. Final preparation checklist

- Open the live site before the interview.
- Open the repository in your editor.
- Keep `Booking.jsx`, `api.php`, `BookingController.php`, the booking migration, and `BookingApiTest.php` in editor tabs.
- Test the live booking page before the interview, but avoid leaving fake bookings in production.
- Practise the 30-second opening and the booking flow more than the minor features.
- Explain the reason behind each choice; do not only name technologies.
- If you do not know an answer, explain how you would investigate it instead of guessing.